

Organização e Recuperação de Dados

Prof. Jonathan

23 de abril de 2026

Conteúdo

1	Memória Secundária	2
1.1	Discos Magnéticos	2
1.2	Custo de Acesso ao Disco	4
1.3	SSD	6
2	Operações em Arquivos	8
2.1	Arquivos	8
2.1.1	Diferentes Visões do Arquivo	8
2.1.2	Conectando Arquivo Lógico ao Físico	8
2.2	Arquivo Lógico	9
2.3	Tipos de Arquivos	9
2.4	Operações em arquivos	10
2.4.1	Criar e abrir arquivos	10
2.4.2	Fechamento de arquivos	11
2.5	Leitura e escrita em arquivos	12
2.5.1	Leitura em modo texto	12
2.5.2	Escrita em modo texto	13
2.5.3	Leitura em modo binário	13
2.5.4	Escrita em modo binário	13
2.5.5	Deteção de fim de arquivo	14
2.6	Posicionamento do Ponteiro de L/E	14
2.7	Arquivos e o S.O	15
2.7.1	Arquivos binários em Python	15
3	Organização em campos e registros	15
3.1	Estrutura de Campos	16
3.1.1	Campos com Tamanho Fixo	16
4	Acesso Sequencial . . .	17
5	Busca Binária	17
5.1	Busca Binária em memória	17
5.2	Busca binária em um arquivo	17
6	(	18

1 Memória Secundária

- Por que discos e fitas são lentos?
 - Por que eles são dispositivos eletromagnéticos que envolvem partes mecânicas, enquanto outros tipos de memória são eletrônicas.
- SSDs não possuem partes mecânicas, por isso são dispositivos secundários mais rápidos
 - Mas ainda são milhares de vezes mais lentos do que a memória RAM
- Fitas são dispositivos seriais, enquanto isso, HDs e SSDs são dispositivos de acesso aleatório

1.1 Discos Magnéticos

- Composição:
 - Um ou mais pratos circulares sobrepostos atravessados por um eixo que os rotaciona
 - Os pratos são construídos com materiais não magnéticos e revestidos por uma cobertura magnética extremamente fina
 - Cabeças de leitura/escrita posicionadas sobre as superfícies do prato são responsáveis pela leitura e gravação dos dados
 - **SEEKING** é um termo importante
- Funcionamento
 - Durante uma leitura/gravação a cabeça permanece parada enquanto o prato gira sob ela
 - * Esta disposição da origem e organização dos dados no prato como um conjunto de anéis concêntricos chamados de trilhas
 - Pulsos magnéticos enviados a uma bobina na cabeça de L/E produzem campos magnéticos que alteram a polaridade de pequenas regiões de uma trilha do prato
 - * A diferença no sentido da polaridade entre cada região codifica a informação binária (0s e 1s) da trilha
 - Durante a leitura, quando as regiões magnetizadas passam sob a cabeça, mudanças de polaridade induzem pulsos elétricos na bobina da cabeça são convertidos novamente para a informação binária equivalente
- Organização dos discos
 - Os dados são armazenados em trilhas sucessivas na superfície de um prato
 - Cada trilha é dividida em setores
 - * Um setor é a menor parte endereçável do disco (512B/4KB)
 - Quando é realizada uma chamada READ() por um bit específico

- * O disco localiza a superfície, a trilha e o setor corretos
- * Os dados de um setor inteiros são copiados para um buffer
- * A partir do buffer, se encontra o byte específico que foi requisitado
- Normalmente os discos vêm setorizados de fábrica (formatação física)
- OBS.: É importante notar que, logicamente, o disco é visto como um vetor contínuo de setores, i.e, a organização física é transparente para o sistema
- Lacunas:
 - * Lacunas entre setores: Evitam que seja imposta uma precisão absurda para a cabeça de L/E
 - * Lacunas entre trilhas: Evitam, ou pelo menos minimizam, erros devido ao desalinhamento da cabeça ou simplesmente à interferência de campos magnéticos
- Em um disco formado por diversos pratos, o conjunto de trilhas na mesma direção (sobrepostas em pratos diferentes) forma um cilindro
 - * Um prato possui o mesmo número de trilhas na parte de cima, e na parte de baixo. (Na hora de contar as trilhas de um prato, se contar só pela parte de cima, lembrar de multiplicar por 2)
- Toda a informação adicional em um cilindro pode ser acessada
- Estimativa de capacidade
 - A quantidade de dados que pode ser armazenada em uma trilha depende do quão densamente os bits podem ser armazenados
 - A densidade depende da qualidade da mídia e da precisão das cabeças de leitura/escrita
 - * Ex.: Um disco rígido de 1TB Seagate 7200RPM ($\pm$ R\$300,00)
 - 16383 trilhas/superfície e 63 setores (de 4k) por trilha (256KB/trilha)
 - Sendo que:
 - * Números de Cilindros = Número de trilhas/superfície
 - * Número de Trilhas do Cilindro = 2x o número de pratos
 - Temos que:
 - * Capacidade da trilha =

$$\text{Número de bytes por setor} \times \text{Números de setor por trilha}$$
 - * Capacidade do cilindro =

$$\text{Capacidade da trilha} \times \text{Números de trilhas do cilindro}$$
 - * Capacidade do disco =

$$\text{Capacidade do cilindro} \times \text{Número de cilindros}$$
 - Exemplo
 - * Propriedades do disco:

- 512 bytes/setor
 - 63 setores/trilha
 - 16 trilhas/cilindro
 - 4080 cilindros
 - * Quantos cilindros são necessários para armazenar um arquivo de 50000 registros de 256 bytes cada
 - R: ...
- Clusters
 - Organização lógica que visa aumentar o desempenho e mantida pelo gerenciador de arquivos, presente no S.O
 - * Quando um arquivo é acessado por uma aplicação é o gerenciador de arquivos quem associa o arquivo lógico às suas posições físicas
 - * O arquivo é visto como uma série de clusters
 - Para fazer esse mapeamento, o gerenciador de arquivos utiliza uma tabela de alocação de arquivos (chamada de File Allocation Table (FAT) em alguns S.Os.)
 - Em vez da tabela de alocação endereçar setores, endereça clusters
 - * Um cluster é um conjunto de um ou mais setores contíguos do disco
 - Todos os setores de um cluster podem ser lidos sem seeks adicionais e sem a necessidade de consultas adicionais à tabela de alocação
 - * A tabela de alocação contém uma lista de todos os clusters de um arquivo, ordenados de acordo com a ordem lógica dos setores que eles contém
 - Cada cluster do disco é usado para um único arquivo, ou seja, em um mesmo cluster não haverá informações sobre mais de um arquivo.
- Custos de acesso a disco
 - Em termos de tempo, o custo de um acesso ao disco é a soma de três tempos:
 - * Posicionamento (Seeking) – Tempo necessário para mover a cabeça de L/E até a trilha correta (Depende da distância a ser percorrida)
 - * Latência (Rotational Delay ou Latency) – Tempo necessário para a cabeça de L/E se posicionar no setor correto.
 - * Transferência (Transfer Time) – Tempo necessário para um byte ser lido na superfície do disco e transferido para o buffer interno da controladora

1.2 Custo de Acesso ao Disco

- Seek Time – Posicionamento
 - É impossível saber exatamente quantas trilhas serão percorridas durante as buscas
 - O que se faz é estimar um tempo médio, assumindo que as posições iniciais e finais para cada acesso são aleatórias
 - Por meio de estudos empíricos, estimou-se que uma busca percorre, em média, 1/3 do total de trilhas, sendo que o tempo gasto para percorrer esse número de trilhas tem sido usado pelos fabricantes como indicador do tempo médio de busca (seek time).

- O que ocorre quando um arquivo está armazenado em cilindros consecutivos e é acessado sequencialmente?
 - * O seek time é reduzido! (Melhor situação)
- O que ocorre quando dois arquivos, localizados em extremos opostos do disco (um no cilindro mais externo e outro no mais interno), são acessados alternadamente?
 - * O seek time é alto! (Pior situação)
- O tempo de seek tende a ser mais caro em sistemas multiusuários em que vários processos concorrem pelo uso do disco
- Latência
 - Tempo gasto para a cabeça de L/E encontre o setor desejado
 - * Estima-se que o tempo medio seja a metade do tempo de uma rotação (na pratica, esse tempo costuma ser menor)
 - * Também chamado de Average Latency
 - * No inicio dos anos 90, com discos de 3600rpm, a metade do tempo de rotação era de 8,3ms
 - * Atualmente os discos de 7200rpm tem latencia media de 4,16ms
 - Em uma leitura/escrita sequencial envolvendo trilhas de um mesmo cilindro, não há latencia na alternância entre as trilhas
 - * Existe apenas um tempo (muito pequeno) de comutação entre as cabeças de leitura/escrita
- Tempo de Transferência
 - Uma vez que a cabeça de L/E está sob o setor desejado, ele pode ser transferido
 - O tempo de transferencia é dado por:
 - * O tempo para transferir uma trilha inteira normalmente é o tempo de rotação
 - * $(n^\circ \text{ bytes transferidos} / n^\circ \text{ bytes na trilha}) \times \text{tempo de rotação}$
 - * Exemplo:
 - Tempo de transferencia de 1KB em disco com 32 setores de 512 bytes por trilha (16.384b) e tempo de rotação de 8,2ms
$$(1024/16384) \times 8,2 = 0,51ms$$
 - Neste exemplo o tempo está expresso em bytes/ms. Os fabricantes costumam utilizar MB/s
- O modo de acesso ao arquivo pode afetar drasticamente os custos de tempo de acesso
 - Dois modos de acesso:
 - * Sequencial: o máximo do arquivo é processado em cada acesso
 - * Aleatório: apenas um registro é processado por vez
 - Exemplo:

- * Determinar o tempo necessário para ler um arquivo com 40000 registros de 256 bytes para cada um
 - Vamos calcular o tempo de leitura do arquivo com acesso sequencial e aleatório e comparar resultados
- * Vamos considerar as seguintes especificações do disco:

Tempo médio de seek	13 ms
Tempo de latência	8.3 ms
Tempo de transferencia	16.7 ms ou 1229 bytes/ms
Bytes por setor	512
Setores por trilha	100
Trilhas por cilindro	12
Trilhas por superfície	1748
Tamanho do cluster	10 setores (5120 bytes)

- * Tamanho do arquivo *Rightarrow* 40.000 registros de 256 bytes
 - Se cada cluster tem 5120 bytes (10 setores), podemos armazenar 20 registros por cluster (5120/256)
 - Será necessário um total de 2000 clusters para armazenar todos os registros (40000/20)
 - Como cada trilha possui 10 clusters (100 setores), serão necessárias 200 trilhas
- * Vamos assumir o pior caso, no qual as 200 trilhas que armazenam o arquivo estão espalhadas aleatoriamente no disco
 - Situação extrema, mas que pode ocorrer em discos no limite da capacidade, especialmente com muitos arquivos pequenos
- * Custo com acesso sequencial
 - Para cada trilha em que são lidos setores consecutivos, o processo de leitura envolve os seguintes custos: Tempo médio de seek, tempo de latência e tempo de transferencia de uma trilha.
 - Para 200 trilhas:

$$(13 + 8,3 + 16,7) \times 200 = 7600ms = 7,6s$$

- * Custo com acesso aleatório
 - Para cada trilha em que são lidos setores consecutivos, o processo de leitura envolve os seguintes custos: Tempo médio de seek, tempo de latência e tempo de transferencia de um cluster.

1.3 SSD

- Solid-state drives(SSD) são compostos por componentes eletrônicos
 - O armazenamento geralmente é feito em módulos de memória flash.
- Devido a essa característica, o custo de acesso a qualquer posição do SSD será o mesmo

- Não há tempo de seek, nem latencia rotacional
- Mas existe uma latência (tempo para se iniciar uma operação) e um tempo de transferencia (throughput)
 - * Esses tempos dependem do tipo de memória usada, da controladora e da interface do SSD
- A menor unidade endereçável de um SSD é uma página.
 - O tamanho de uma página muda de um SSD para outro, variando entre 2KB e 16KB.
 - Assim como acontece com os setores de um disco, a pagina é a menor unidade de leitura e escrita.
- As paginas são agrupadas em blocos.
 - O tamanho dos blocos tambem varia de um SSD para outro, entre 256KB e 4MB
 - * Por exemplo, o Samsung SSD 840 EVO tem blocos de 2048KB (256 páginas de 8KB cada)
- Diferentemente dos setores de um HD, as paginas de um SSD não podem ser sobrescritas.
 - Quando uma alteração é necessaria, a pagina é copiada para um buffer, alterada a escrita em uma nova página livre.
 - A página antiga é marcada como "velha" e ficará assim até o seu bloco ser apagado e a página fique livre novamente.
- Essa operação faz com que as escritas sejam mais lentas que as leituras.
- Alem disso, apenas blocos podem ser apagados.
 - Para que um bloco com alguma página "velha" seja apagado, o coletor de lixo copiará as paginas ativas para um novo bloco, deixando o bloco pronto para ser apagado por completo.
 - Uma vez apagado, as páginas do bloco ficam livres novamente.
- Além de não poderem ser sobrescritas, memórias flash também têm um numero máximo de escritas possiveis.
 - Elas desgastam conforme são escritas e apagadas
 - Por conta disso, a controladora distribuirá o uso dos blocos de memoria para que o desgaste não seja maior em uma região do que em outra.
- Todos esses fatores fazem com que as controladoras sejam complexas e adicionem custo nas operações em termos de tempo, especialmente nas escritas.
- Ainda assim, os SSDs são centenas de vezes mais rapidos do que os HDs, especialmente quando se trata de acesso aleatório.

Independente do tipo de dispositivo usado para como memória secundária, ele será o gargalo do sistema.

- Gargalo → Quando um dispositivo mais lento afeta o desempenho de outros mais rápidos, se tornando um fator limitante do sistema.

Por isso é importante que acessos à memória secundária sejam feitos de forma eficiente.

2 Operações em Arquivos

2.1 Arquivos

- Um arquivo é uma estrutura criada em uma unidade de armazenamento secundário
 - Permite o armazenamento permanente dos dados, ao contrário das variáveis, que são armazenadas em RAM
- Do ponto de vista físico, um arquivo corresponde a uma sequência de bytes armazenados em um dispositivo secundário
 - Mas os aspectos físicos dos arquivos são transparentes para as aplicações

2.1.1 Diferentes Visões do Arquivo

- Arquivo físico e arquivo lógico
 - Arquivo físico: é o arquivo do ponto de vista do armazenamento
 - * Corresponde a uma sequência particular de bytes armazenados no dispositivo
 - Arquivo lógico: é o arquivo do ponto de vista da aplicação que o acessa
 - * Cada arquivo manipulado pela aplicação é tratado como um canal de comunicação estabelecido entre a aplicação e o arquivo físico
- O sistema operacional é quem faz a interface entre o arquivo lógico e o arquivo físico.
- Apesar de um dispositivo secundário poder conter milhares de arquivos físicos, as aplicações, em geral, só podem abrir um número pré-definido de arquivos lógicos de forma simultânea
- Um arquivo lógico pode estar associado a um arquivo físico ou a outros dispositivos de E/S, como o teclado (stdin) e o vídeo (stdout e stderr)
- Para os sistemas operacionais, um dispositivo de E/S é visto da mesma forma que um arquivo!

2.1.2 Conectando Arquivo Lógico ao Físico

- A aplicação solicita que um nome lógico seja associado a um arquivo físico específico e o S.O estabelece a associação

2.2 Arquivo Lógico

- Do ponto de vista da aplicação, um arquivo é uma entidade lógica vista como uma sequência de bytes
 - Podemos enviar bytes para o arquivo indefinidamente
 - De forma análoga, podemos ler bytes do arquivo enquanto houverem bytes a serem lidos
- Para todo arquivo lógico, é mantido um ponteiro de L/E que indica a posição do próximo byte a ser lido/escrito
 - Esse ponteiro se move automaticamente conforme operações de leitura e escrita são realizadas e também pode ser movido por comandos de reposicionamento.
- Quando um arquivo é aberto, o ponteiro de L/E é posicionado no início (byte 0)
- Se for feita a leitura de 9 bytes, o ponteiro de L/E se move automaticamente para a próxima posição de L/E (byte 9)
- Se for feita uma escrita de 8 bytes na sequência, o ponteiro de L/E se moverá para a próxima posição de L/E (byte 17)
- O arquivo lógico é identificado pelo S.O por um número inteiro chamado de descritor
- É comum que as linguagens encapsulem o descritor do arquivo em estruturas mais complexas, que armazenam outras informações úteis para a manipulação do arquivo
 - O tipo dessa estrutura descritora é dependente da linguagem
 - Em Python, essas informações ficam em um objeto de arquivo

2.3 Tipos de Arquivos

- Basicamente são dois tipos de arquivos
 - Texto
 - * Os caracteres são armazenados sequencialmente
 - * É possível determinar o primeiro, segundo, terceiro, n caracteres que compõem o arquivo
 - * Nesse tipo de arquivo, a sequência 123 corresponde à string "123"
 - Binário
 - * Formado por uma sequência de bytes sem correspondência direta com um tipo de dado
 - * Cabe ao programador fazer essa correspondência quando lê e escreve nesses programas
 - * Nesse tipo de arquivo, o valor 123 será gravado como um número binário com uma quantidade pré-definida de bytes
 - Exemplo:
 - * As figuras mostram um arquivo binário e um arquivo texto em um editor hexadecimal

2.4 Operações em arquivos

- Basicamente as operações em arquivos são as seguintes:
 1. Criar um arquivo
 2. Abrir um arquivo
 3. Ler de um arquivo
 4. Escrever em um arquivo
 5. Movimentar o ponteiro de L/e
 6. Fechar um arquivo
- Veremos como essas operações são feitas em Python
- As funções da linguagem Python para manipular arquivos são implementadas de duas formas
 - Como função nativa
 - Como métodos de objetos de arquivo
- A função `open` é nativa e utilizada para abrir/criar um arquivo retornando um objeto de arquivo
 - A classe desse objeto varia de acordo com o tipo de arquivo a ser manipulado (leitura/escrita textual, leitura/escrita binária, etc.)
 - A partir do objeto de arquivo é possível acessar métodos para leitura, escrita, posicionamento, etc.
- A associação entre arquivo lógico e físico é feita no momento da abertura/criação do arquivo.
 - O modo como associação se dará depende do parametro utilizado para especificar o modo de abertura
 - O modo de abertura também especifica se um novo arquivo será criado ou se um arquivo existente será aberto

2.4.1 Criar e abrir arquivos

- `def open(filename:str, mode:str) -> file object`
 - `filename` é uma string contendo o nome do arquivo físico
 - `mode` também é uma string e define o modo de abertura
 - **Caso a operação falhe, um erro ocorrerá e abortará a execução!**
 - Exemplo: `arq = open('meuarq.txt', 'w')`
 - Flags/modos da função `open()`

Caractere	Descrição do comportamento
'r'	Abre para leitura. O arquivo deve existir. É o modo padrão.
'w'	Abre para escrita. Se o arquivo existir ele será truncando. Senão, será criado.
'x'	Abre um novo arquivo para escrita. O arquivo não deve existir.
'a'	Abre para escrita no final do arquivo. Operações de reposicionamento do ponteiro de L/E são ignoradas. Se não existir, o arquivo será criado.
't'	Abre em modo texto. Admite apenas dados do tipo <code>str</code> . É o modo padrão.
'b'	Abre em modo binário. Admite apenas dados do tipo <code>bytes/bytearray</code> .
'+'	Altera o comportamento da flag inicial para permitir leitura e escrita.

- * As flags 't', 'b' e '+' só podem ser utilizadas em conjunto de outras!
- * Por padrão os arquivos são abertos em modo texto (flag 't')

- Como tratar um erro na abertura do arquivo?

- Em Python, erros de execução são chamados de exceções
- As exceções podem ser capturadas e tratadas utilizando a instrução `try-except`

```
try:
    ''' código a ser executado '''
except:
    ''' código a ser executado caso uma exceção ocorra '''
```

- Exemplo:

```
try:
    arq = open('meuArq.txt', 'r')
    print('abertura OK')
except:
    print('Erro na abertura')
    exit()
```

2.4.2 Fechamento de arquivos

- Uma vez que o arquivo foi aberto, seu fechamento se dá pela chamada do método `close()`
 - Todos os objetos de arquivo possuem esse método
 - Encerra a associação entre o arquivo lógico e o físico
 - O descritor fica disponível para uso por outro arquivo
 - Garante que todas as informações do arquivo foram atualizadas (o fechamento garante que o conteúdo bufferizado foi enviado para o dispositivo físico)

- O S.O normalmente fecha qualquer arquivo que ficou aberto quando um programa termina corretamente, mas o ideal é fechar arquivos que não estão mais sendo utilizados
 - * Previne a perda de dados bufferizados no caso de erro
 - * Libera a estrutura descritora para novos arquivos
- Exemplo:
 - ```
arq = open('meuarq.txt', 'r')
''' comandos '''
arq.close()
```
- Quando o arquivo é aberto em um comando `with`, não é necessário o fechamento explícito
  - ```
with open('meuarq.txt', 'r') as arq
''' comandos '''
```
 - * Nesse caso, o objeto de arquivo `arq` só existirá no escopo do comando `with`

2.5 Leitura e escrita em arquivos

- Em Python, as funções de leitura e escrita são métodos do objeto de arquivo
 - Portanto, dependem do tipo do objeto
- Arquivos em modo texto são representados por objetos do tipo **TextIOWrapper**
 - Só admitem leitura e escrita de strings, i.e., objetos do tipo `str`
- Arquivos em modo binário são representados por objetos **BufferedReader** (leitura), **BufferedWriter** (escrita) e **BufferedRandom** (leitura e escrita)
 - Só admitem leitura e escrita de objetos do tipo `bytes` ou `bytearray`
 - * `bytes` são imutáveis e `bytearray` são mutáveis – ambos representam sequências de bytes

2.5.1 Leitura em modo texto

- `def read(size=-1) -> str`
 - Lê e retorna no máximo `size` caracteres como uma única string
 - Se `size` for negativo ou `None`, lê até o fim do arquivo (EOF), i.e., retorna todo o conteúdo do arquivo como uma única string
- `def readline(size=-1) -> str`
 - Lê até uma quebra de linha ou EOF e retorna o conteúdo como uma única string
 - Se o ponteiro L/E já estiver EOF, retorna uma string vazia
 - Se `size` for especificado, no máximo `size` caracteres serão lidos

- `def readlines(hint=-1) -> list[str]`
 - Retorna as linhas do arquivo como uma lista de strings
 - Se `hint` for especificado, linhas serão lidas até o maximo de `hint` caracteres no total

2.5.2 Escrita em modo texto

- `def write(s) -> int`
 - Escreve a string `s` no arquivo
 - Retorna o numero de caracteres escritos
- `writelines(lines) -> None`
 - Escreve uma lista de strings
 - Não são adicionados separadores entre as strings, então é comum que cada string contenha um caractere separador (p.e., `'\n'`) no final.

2.5.3 Leitura em modo binário

- `def read(size=-1) -> bytes`
 - Lê e retorna até `size` bytes
 - Se `size` for negativo ou `None`, o conteudo do arquivo será lido até EOF e retornado
 - Um objeto `bytes` vazio(`b''`) é retornado se o ponteiro estiver em EOF
- `def readinto(b) -> int`
 - Lê bytes para uma variavel `b`, pré-alocada como um `bytearray`, e retorna o numero de bytes lidos
- `def readlines(hint=-1) -> list[bytes]`
 - Retorna o conteudo do arquivo como uma lista de linhas
 - Considera o valor `b'\n'` como delimitador de linha
 - Se `hint` for especificado, linhas serão lidas até o maximo de `hint` bytes no total

2.5.4 Escrita em modo binário

- `def write(b) -> int`
 - Escreve o objeto `b` do tipo `bytes` ou `bytearray` no arquivo
 - Retorna o numero de bytes escritos (sempre igual ao comprimento de `b` em bytes, pois uma falha na escrita irá gerar uma exceção)
- `def writelines(lines) -> None`
 - Escreve uma lista de linhas no arquivo
 - Não são adicionados separadores entre as linhas, então é comum que cada linha seja adicionada de um caractere separador (p.e., `b'\n'`)

2.5.5 Detecção de fim de arquivo

- Durante a leitura de um arquivo, o S.O monitora a posição do ponteiro de L/E do arquivo
- Em Python, não existe uma função específica para isso, mas as funções de leitura detectam o fim de arquivo
 - Nesse caso, as funções de leitura retornam um objeto vazio, seja uma string (") ou um byte (b")

2.6 Posicionamento do Ponteiro de L/E

- Funções de leitura e escrita movimentam o ponteiro de L/E do arquivo de forma automática
 - O ponteiro de L/E sempre avança quando lê/escreve
- Podemos movimentar o ponteiro de L/E para uma posição específica do arquivo usando o método `seek`
 - Chamamos o tamanho do deslocamento a ser realizado de `byte-offset`
 - Quando esse deslocamento é contado a partir do início do arquivo, podemos pensar no offset como um "endereço" para onde se deseja mover o ponteiro de L/E
- `def seek(offset, whence = os.SEEK_SET) -> int`
 - Posiciona o ponteiro de L/E no byte offset calculado a partir da origem dada (whence)
 - Retorna o valor absoluto da nova posição
 - O parâmetro whence aceita os seguintes valores:
 - * `os.SEEK_SET` ou 0 → Início do arquivo. É o valor padrão.
 - * `os.SEEK_CUR` ou 1 → Posição atual do ponteiro de L/E.
 - * `os.SEEK_END` ou 2 → Fim do arquivo.
- Arquivos em modo texto só aceitam see com offsets positivos a partir de `os.SEEK_SET`
 - Nesse caso, se whence for `os.SEEK_CUR` ou `os.SEEK_END`, o offset deverá ser 0
- Como voltar o ponteiro de L/E para o início?
 - `arq.seek(0)`
- Como posicionar o ponteiro de L/E no fim?
 - `arq.seek(0, os.SEEK_END)`
- Como saber em qual posição o ponteiro de L/E está?
 - Todo objeto de arquivo possui um método `tell`
 - `def tell() -> int`
 - * Retorna a posição atual do ponteiro de L/E como um número inteiro

2.7 Arquivos e o S.O

- Marcação de fim de linha
 - Dependendo do S.O utilizado, a representação de fim de linha muda
 - * DOS/Windows utiliza dois bytes
 - Par CR-LF (decimal ASCII 13 e 10 → hex 0D 0A)
 - * Unix/Linux utiliza um byte
 - LF (decimal ASCII 10 → hex 0A)
 - Para vermos essa marcação precisamos usar um editor hexadecimal

2.7.1 Arquivos binários em Python

- Os arquivos binários em Python realizam escrita e leitura apenas com os tipos bytes e bytearray
- Alguns tipos fornecem métodos de conversão prontos para os tipos binários
 - Conversão de inteiros
 - * `<int>.to_bytes(n)` → converte inteiro para n bytes
 - * `int.from_byte(<bytes>)` → converte bytes para inteiros
 - Conversão de strings:
 - * `<str>.encode()` → converte string para bytes
 - * `<byte>.decode()` → converte bytes para string

3 Organização em campos e registros

- Queremos criar um programa que permita cadastrar pessoas
 - Para cada pessoa, armazenaremos as seguintes informações
 - * Sobrenome, Nome, Endereço, Cidade, Estado e CEP
 - Por exemplo:
 - * Alan Silva, Rua Tiete 123, Maringá, PR, 87100
 - * Andre Flores, Rua Braga 34, Sarandi, PR, 87111
- Cada uma dessas informações constitui um agregado com um significado próprio, como "Alan" ou "Rua Braga 34"
- Chamamos esses agregados de campos e eles são as menores unidades de formação com significado dentro de um arquivo
- Considere o pseudocódigo abaixo para criar o nosso cadastro de pessoas como um fluxo de bytes

```

PROGRAMA escreve_fluxo:
    receba o nome do arquivo a ser criado e abra o arquivo com o nome
    lógico SAIDA
    receba o SOBRENOME
    enquanto comprimento(SOBRENOME) > 0:
        receba NOME, ENDEREÇO, CIDADE, ESTADO e CEP
        escreva SOBRENOME no arquivo SAIDA
        escreva NOME no arquivo SAIDA
        escreva ENDEREÇO no arquivo SAIDA
        escreva CIDADE no arquivo SAIDA
        escreva ESTADO no arquivo SAIDA
        escreva CEP no arquivo SAIDA
        receba o SOBRENOME
    fim /* enquanto */
    feche SAIDA
fim PROGRAMA

```

- Supondo que cadastramos os dados dos exemplos anteriores usando o nosso programa:

Arquivo gerado:
SilvaAlanRuaTiete123MaringaPR87100FloresAndreRuaBraga34SarandiPR87111

- Perdemos a integridade das unidades de informação das entradas de dados
- Não conseguiremos mais separar um campo de outro

3.1 Estrutura de Campos

- Existem várias formas de adicionar estrutura aos arquivos para manter a integridade dos campos
 1. Definir campos de tamanhos fixos
 2. Iniciar cada campo com um indicador de tamanho
 3. Separar os campos por um delimitador
 4. Usar uma expressão do tipo *palavra-chave=valor* para identificar cada campo e seu conteúdo

3.1.1 Campos com Tamanho Fixo

- Força os dados em campos de tamanho fixo
- Aumento do tamanho do arquivo: normalmente se aloca mais espaço que o necessário
- Possível perda de informação → pelo limite imposto ao tamanho dos campos, os valores dos campos podem ser truncados (cortados)
- Indicado quando o tamanho dos valores a serem armazenados for fixo ou com pouca variação

- Facilita a implementação em linguagens que utilizam tipos com tamanhos pré-definidos

4 Acesso Sequencial ...

5 Busca Binária

- ...
- ...

5.1 Busca Binária em memória

- Função que recebe um valor a ser buscado e uma lista ordenada de valores
- Retorna ...

5.2 Busca binária em um arquivo

- Pre requisitos para BB em arquivo
 - Requer registros de tamanho fixo
 - O arquivo deve estar ordenado em ordem crescente da chave de busca
- Complexidade
 - Em um arquivo de n registros
 - * Busca binária: $O(\log_2 n)$, pior caso $\lfloor \log_2 n \rfloor$
 - Em cada chute elimina-se metade das possibilidades
- A busca binária é mais eficiente do que a sequencial, porém o arquivo deve estar ordenado por chave
 - Custo de ordenar e manter o arquivo ordenado após novas inserções
 - * A cada inserção deve-se reordenar o arquivo ou fazer a inserção no arquivo em memória e regravar o disco
- Uma primeira solução $\rightarrow$ Ordenação interna
 - Só é factível se o arquivo for pequeno e couber na memória
 1. Ler todo o arquivo do disco para a RAM (acesso sequencial)
 2. Ordenar os registros em RAM usando um algoritmo de ordenação

6 (

)

- Objetivo
 - Processar de forma coordenada duas ou mais listas sequenciais para produzir uma lista unica como saida
 - * Por ex., duas ou mais listas de chaves primarias